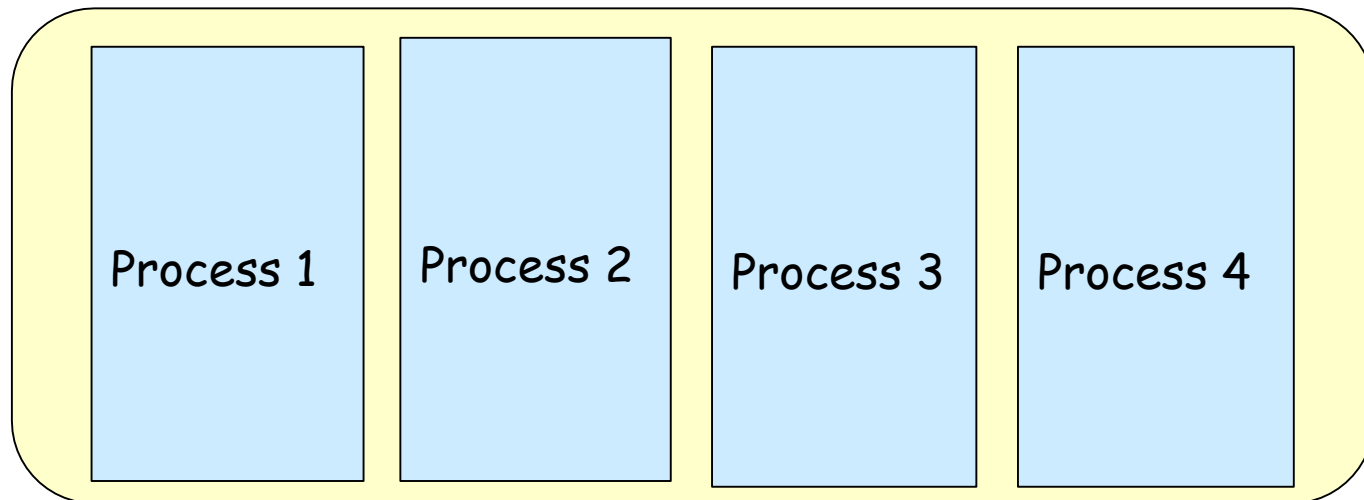


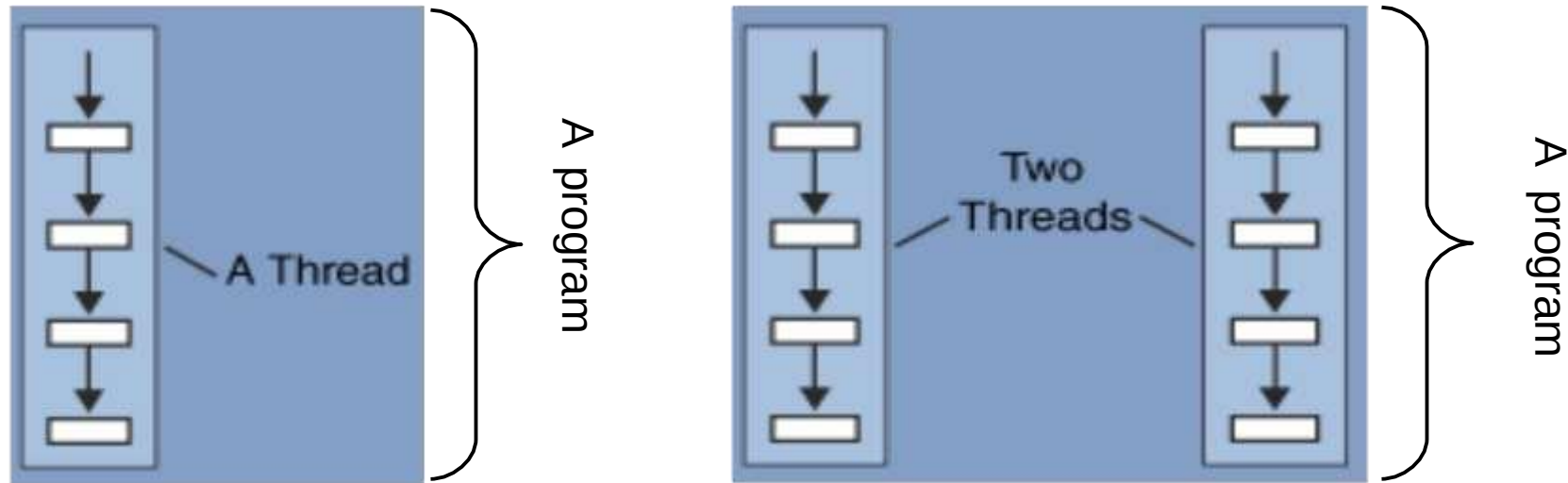
Process

- An executing instance of a program is called process.
- Also known as “Heavyweight process”.
- It has own address space.
- For example while writing java program in editor we can run MP3 player. At the same time we can download file from net. All these task are executing simultaneously and independent of each other.



What is Thread?

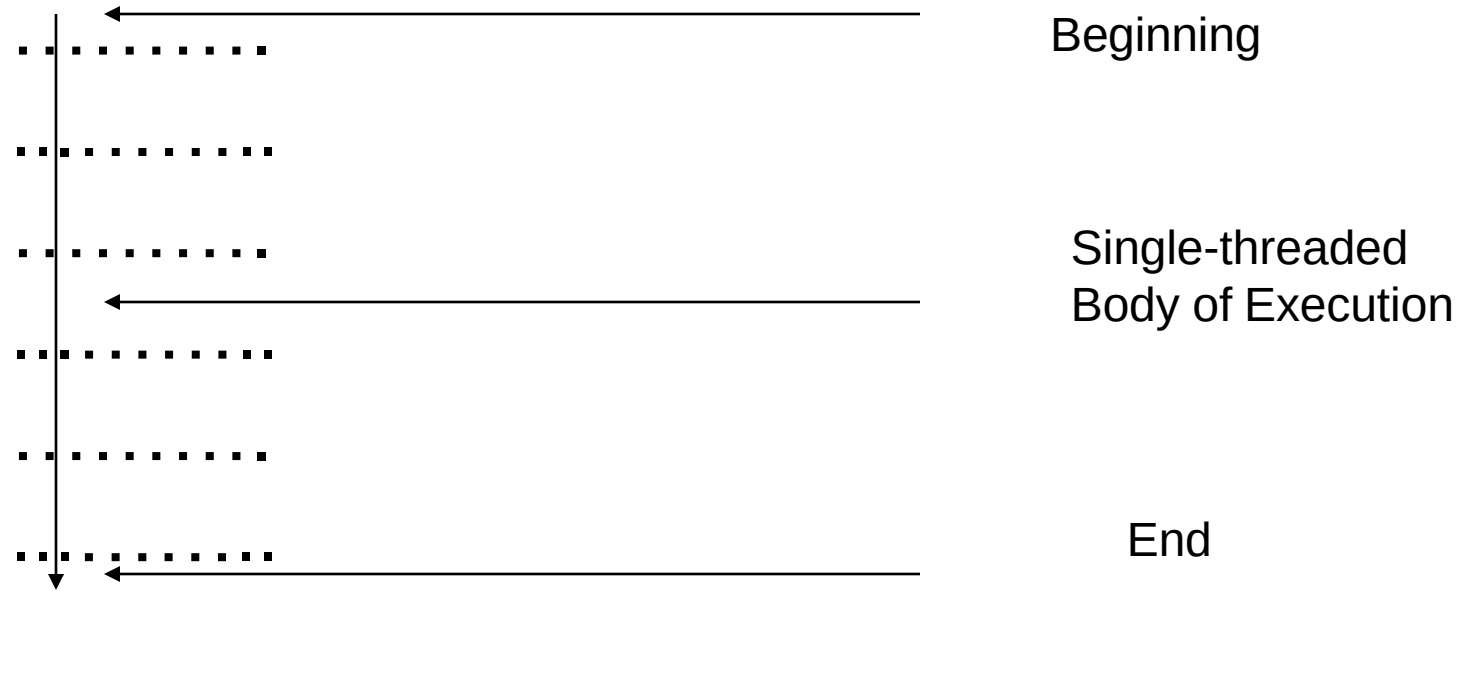
- ❑ A thread is a single sequential flow of execution within a program.
- ❑ A thread is a subset of process.
- ❑ Thread is considered as “Light Weight Process” because it use less resource than process.
- ❑ They share the address space of the process but process have their own address space.
- ❑ JVM create the thread whose task is to define the main thread.



A Single Threaded Program

```
class ABC
```

```
{
```



```
}
```

Application Thread

When we execute an application:

1. The JVM **creates** a Thread object whose task is defined by the **main()** method .
2. The JVM **starts** the thread .
3. The thread **executes** the statements of the program one by one.
4. After executing all the statements, the method returns and the **thread dies**.

Process

- ❑ Process can be divided into multiple threads
- ❑ Each process has its own memory space
- ❑ It is difficult to create a process

Thread

- ❑ Threads cannot be sub divided.
- ❑ Threads of the same process share a common memory space.
- ❑ It is easy to create a thread

Difference between Process and Threads

MULTITHREADING

The process of executing multiple threads simultaneously is known as multithreading.

Example

- ❑ Games are very good examples of threading. You can use multiple objects in games like cars, motor bikes, animals, people etc. All these objects are nothing but just threads that run your game application.
- ❑ In word processor one thread take the input from keyboard and another thread check the spelling mistakes and third thread count the words.
- ❑ Railway ticket reservation system where multiple customers accessing the server.
- ❑ Multiple account holders accessing their accounts simultaneously on the server. When you insert a ATM card, it starts a thread for perform your operations.

Advantages of Multithreading

- ❑ The main advantage is proper utilization or max utilization is possible.
- ❑ Decreased cost of maintenance.
- ❑ Time saving.
- ❑ Improves the performance of complex applications.
- ❑ Parallelize tasks.

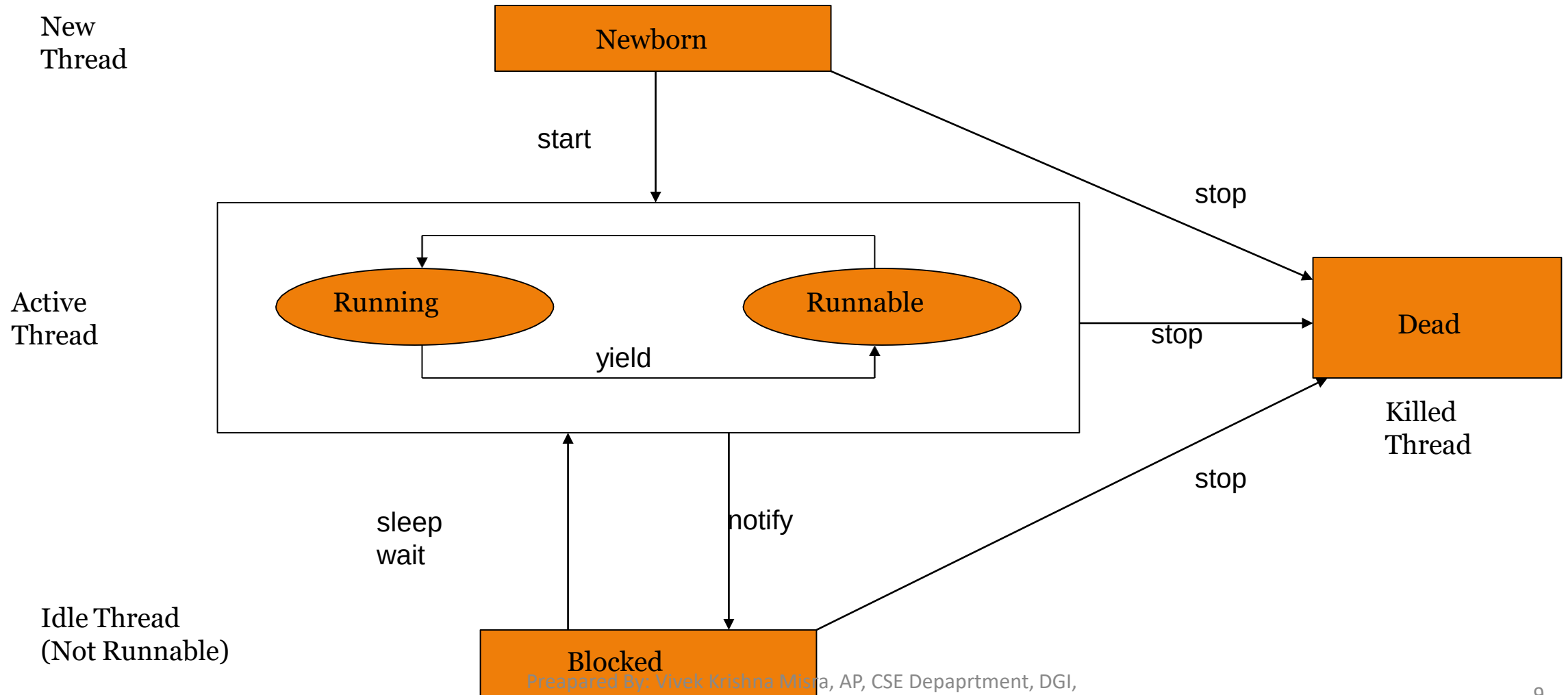
Disadvantage of Multithreading

- ❑ Complex debugging and testing processes.
- ❑ Increased difficulty level in writing a program.
- ❑ **Unpredictable results.**

Thread Life Cycle

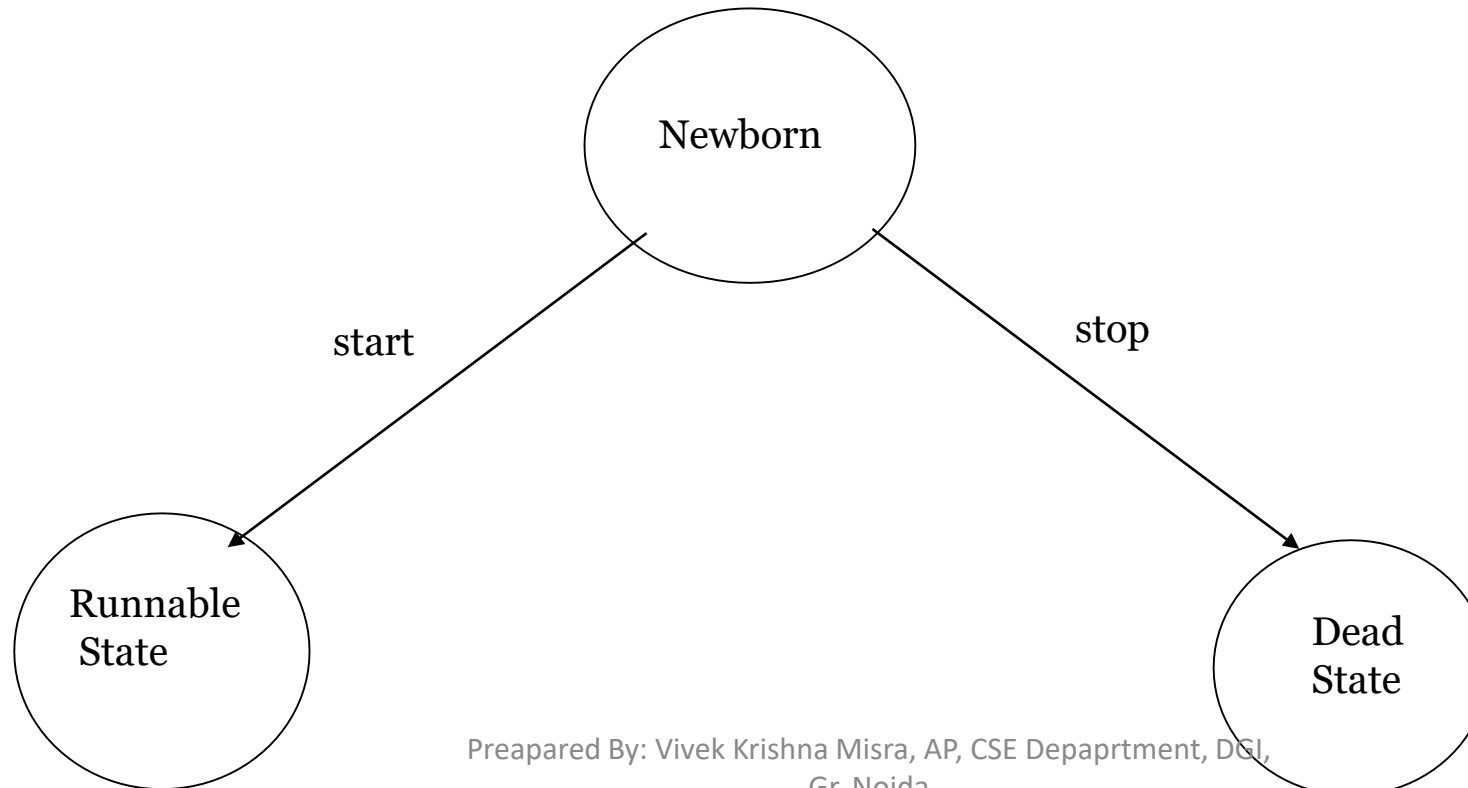
- ❑ During the life time of a thread, there are many states it can enter. They include:
 1. Newborn state
 2. Runnable State
 3. Running State
 4. Blocked State
 5. Dead State

State diagram of a Thread



1. Newborn State

- ❑ When we create a thread object, the thread is born and is said to be in newborn state. The thread is not yet scheduled for running. At this state, we can do only one of the following with it:
 - ❑ Schedule it for running using start() method.
 - ❑ Kill it using stop() method.



2. Runnable state (start())

- ❑ The runnable state means that the thread is ready for execution and is waiting for the availability of the processor.
- ❑ That is, the thread has joined the queue of threads that are waiting for execution.
- ❑ If all threads have equal priority, then they are given time slots for execution in round robin fashion. i.e. first-come, first-serve manner.

3. Running State

- ❑ Running means that the processor has given its time to the thread for its execution.
- ❑ The thread runs until it relinquishes control on its own or it is preempted by a higher priority thread.

4. Blocked State

- ❑ A thread is said to be blocked when it is prevented from entering into the runnable state and subsequently the running state.
- ❑ This happens when the thread is sleeping, or waiting in order to satisfy certain requirements.
- ❑ A blocked thread is considered “not runnable” but not dead and therefore fully qualified to run again.

5. Dead State

- ❑ A running thread ends its life when it has completed executing its run() method. It is a natural death.
- ❑ However, we can kill it by sending the stop message to it at any state thus causing a premature death to it.
- ❑ A thread can be killed as soon it is born, or while it is running, or even when it is in “not runnable” (blocked) condition.

Threads Methods

❖ Thread Methods :

currentThread

start

run

stop

getName

setName

sleep

setpriority

getpriority

wait

join

isAlive

currentThread() Method

This method return complete information about the current thread like it return name of current thread, name of group of current thread, priority number of current thread.

EXAMPLE

```
class Demo
```

```
{  
    public static void main(String args[])  
    {  
        System.out.println("current thread information is");  
        System.out.println(Thread.currentThread());  
    }  
}
```

Output

current thread information is

Thread[main,5,main]

↓ ↓ ↓
Current thread Priority no Group thread

getName() Method

It is a sub method of current thread method and it return name of current thread.

EXAMPLE

```
class Demo
{
    public static void main(String args[])
    {
        System.out.println("name of current thread ");
        System.out.println(Thread.currentThread().getName());
    }
}
```

output

name of current thread
main

setName() Method

It is a child method of current thread method and it is use to set name of current thread.

EXAMPLE

```
class Demo
{
    public static void main(String args[])
    {
        Thread.currentThread().setName("Parent");
        System.out.println("name of current thread ");
        System.out.println(Thread.currentThread().getName());
    }
}
```

Output

name of current thread
Parent

getPriority() Method

It is a child method of current thread and this method return the priority number of the current thread.

EXAMPLE

```
class Demo
{
    public static void main(String args[])
    {
        System.out.println("current thread priority ");
        System.out.println(Thread.currentThread().getPriority());
    }
}
```

Output

current thread priority
5

setPriority() Method

It is a child method of current thread method and this method is used to set the priority number of current thread.

EXAMPLE

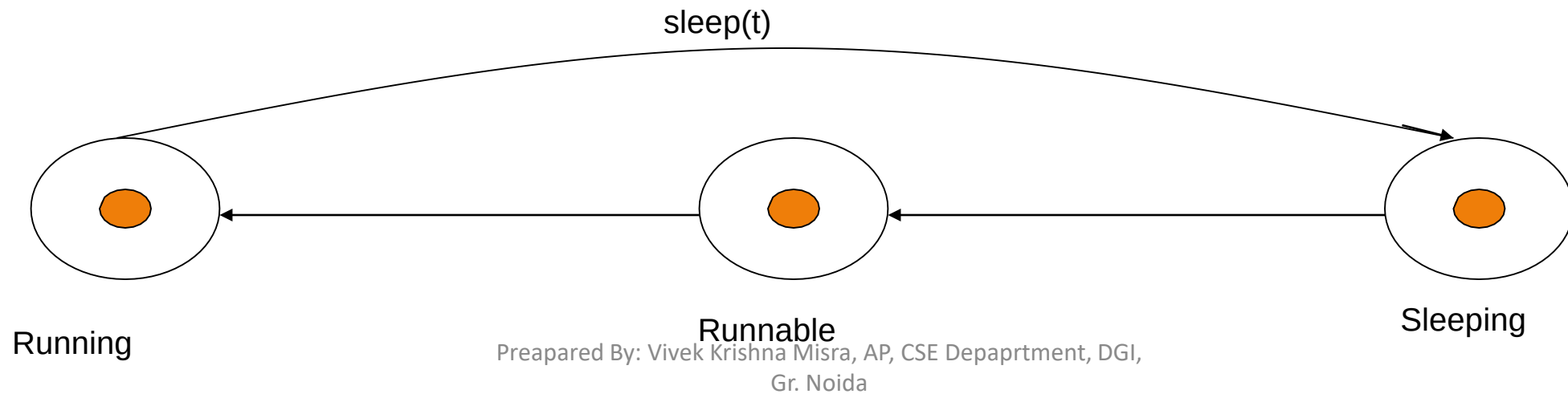
```
class Demo
{
    public static void main(String args[])
    {
        Thread.currentThread().setPriority(9);
        System.out.println("current thread priority ");
        System.out.println(Thread.currentThread().getPriority());
    }
}
```

Output

current thread priority
9

Sleep() Method

This method is used to block the current thread until the given sleep time is not complete.



EXAMPLE

```
class Demo
{
    public static void main(String
args[])
    {
        int i;
        for(i=1;i<=5;i++)
        {
            System.out.println(i);
```

```
        try
        {
            Thread.sleep(2000); //2
sec//
        }
        catch(Exception e)
        {
        }
    }
}
```

Output

1
2
3
4
5

Thread Declaration

 *Thread can be declared in two ways :*

- ☐ *Create a class that **extends** the **Thread** class*
- ☐ *Create a class that **implements** the **Runnable** interface*

1st way:



Create a class by **extending Thread** class and **override run()** method:

```
class MyThread extends Thread
{
    public void run()
    {
        // thread body of execution
    }
}
```

Create a thread:

```
MyThread thr1 = new MyThread();
```

Start Execution of threads:

```
thr1.start();
```

2nd way:

Create a class that **implements** the interface **Runnable** and override run() method:

```
class MyThread implements Runnable
{
    ....
    public void run()
    {
        // thread body of execution
    }
}
```

Creating Object:

```
MyThread myObject = new MyThread();
```

Creating Thread Object:

```
Thread thr1 = new Thread( myObject );
```

Start Execution:

```
thr1.start();
```

Example of extends Thread class

```
class MyThread extends Thread
{
public void run()
{
for(int i=0;i<10;i++)
{
System.out.println("child thread");
}
}
}
```

Job of thread
executed by
child thread

```
class Demo
{
public static void main(String args[])
{
MyThread t=new MyThread(); //main thread creating child thread obj//
t.start(); //starting of thread//
//now there is two thread main and child thread//
for(int i=0;i<10;i++)
{
System.out.println("main thread");
}
}
}
```

Job of thread
executed by
main thread

Example of implements Runnable interface

- class MyThread1 implements Runnable

- {
- public void run()
- {
- for(int i=0;i<5;i++)
- {
- System.out.println("child thread "+i);
- }
- }
- }

Job of thread
executed by child
thread

class Demo

```
{
    public static void main(String args[])
    {
        MyThread1 t1=new MyThread1(); //main thread create runnable obj//
        Thread obj1=new Thread(t1);
        obj1.start();
        for(int i=0;i<5;i++)
        {
            System.out.println("main thread "+i);
        }
    }
}
```

Job of thread
executed by main
thread

F:\New folder\java>java Demo

main thread 0
child thread 0
child thread 1
main thread 1
child thread 2
main thread 2
child thread 3
main thread 3
child thread 4
main thread 4

F:\New folder\java>java Demo

main thread 0
main thread 1
main thread 2
child thread 0
main thread 3
child thread 1
main thread 4
child thread 2
child thread 3
child thread 4

F:\New folder\java>java Demo

main thread 0
child thread 0
child thread 1
child thread 2
main thread 1
child thread 3
main thread 2
child thread 4
main thread 3
main thread 4

F:\New folder\java>java Demo

main thread 0
main thread 1
main thread 2
child thread 0
main thread 3
child thread 1
main thread 4
child thread 2
child thread 3
child thread 4

Difference between extends Thread class and implements Runnable

- **Inheritance Option:** The limitation with "extends Thread" approach is that if you extend Thread, you cannot extend anything else. Java does not support multiple inheritance. In reality, you do not need Thread class behavior, because in order to use a thread you need to instantiate one anyway.
On the other hand,
Implementing the Runnable interface gives you the choice to extend any class you like, but still define behavior that will be run by separate thread.
- Another difference between Thread and Runnable comes from the fact that you are extending Thread class just for run() method but you will get overhead of all other methods which come from Thread class. So, if your goal is to just write some code in run() method for parallel execution then use Runnable instead of extending Thread class.
- When you **extends Thread** class, each of your thread creates unique object and associate with it.
- When you **implements Runnable**, it shares the same object to multiple threads.
- When you **implements Runnable**, you can save a space for your class to extend any other class in future or now.

stop() Method

This method stops a thread completely.

EXAMPLE

```
class Thread1 extends Thread
{
    public void run()
    {
        for(int i=0;i<5;i++)
        {
            if(i==2)
                stop();
            System.out.println("Thread1:"+i);
        }
    }
}
```

```
class Thread2 extends Thread
{
    public void run()
    {
        for(int j=0;j<5;j++)
        {
            if(j==3)
                stop();
            System.out.println("Thread 2:"+j);
        }
    }
}
```

Prepared By: Vivek Krishna Misra, AP, CSE Department, DGI,
Gr. Noida

```
class Demo
{
    public static void main(String args[])
    {
        Thread1 t1=new Thread1();
        Thread2 t2=new Thread2();
        t1.start();
        t2.start();
    }
}
```

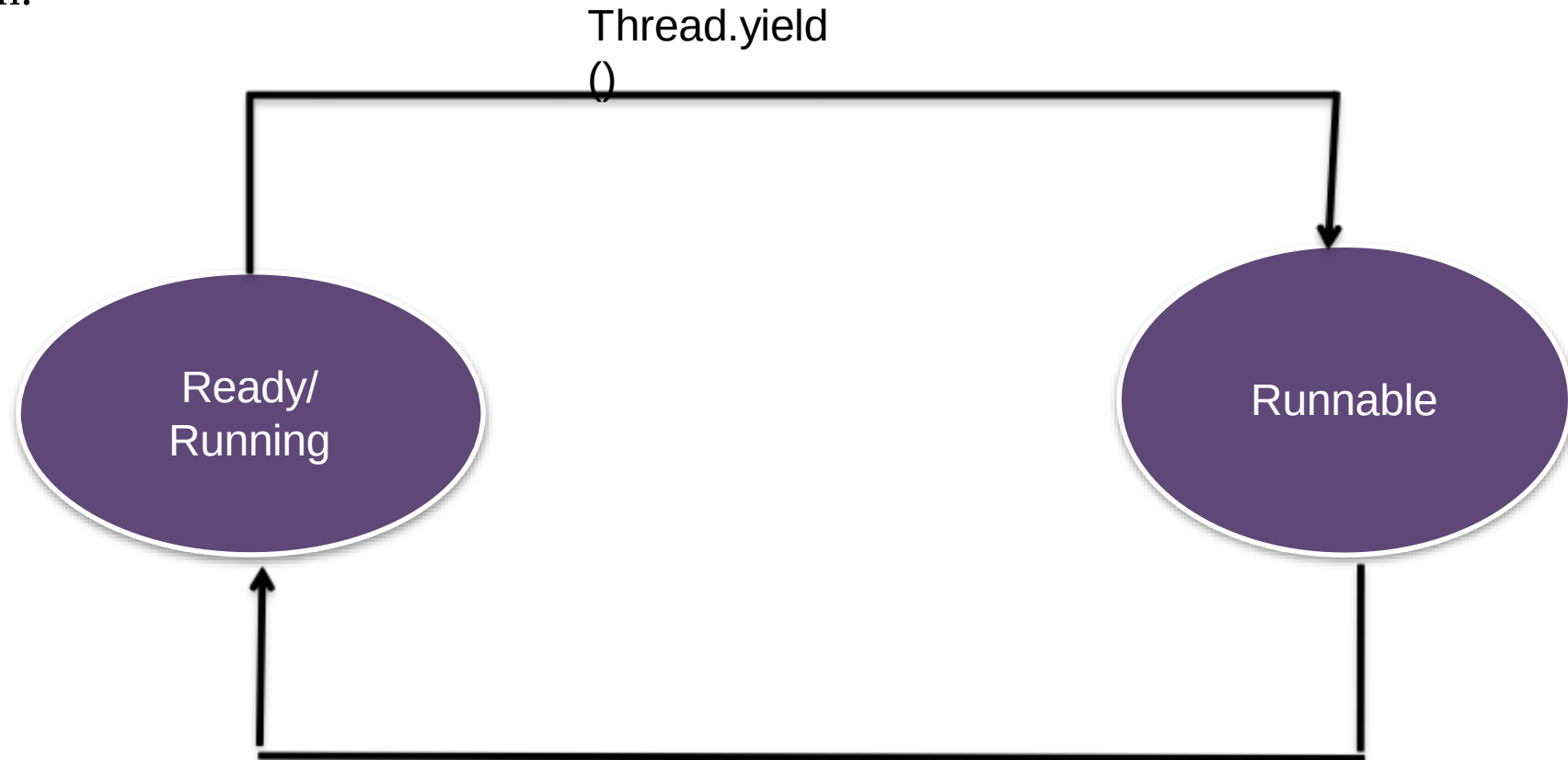
output

Thread 1:0
Thread 1:1
Thread 2:0
Thread 2:1
Thread 2:2

yield() Method

yield() method causes to pause current executing thread for giving the chance to remaining waiting threads of same priority.

If there are no waiting threads or all waiting threads have low priority then the same thread will continue its execution once again.



EXAMPLE

```
class Thread1 extends Thread
{
public void run()
{
for(int i=0;i<5;i++)
{
if(i==2)
yield();
System.out.println("Thread 1:"+i);
}
System.out.println("Exit from thread1");
}
}
```

```
class Thread2 extends Thread
{
public void run()
{
for(int j=0;j<5;j++)
{
System.out.println("Thread2:“
                        +j);
}
System.out.println("Exit
                    from thread2");
}
}
```

```
class Demo
{
public static void main(String args[])
{
Thread1 t1=new Thread1();
Thread2 t2=new Thread2();
t1.start();
t2.start();
}
}
```

Output

Thread 1:0

Thread 1:1

Thread 2:0

Thread 2:1

Thread 2:2

Thread 2:3

Thread 2:4

Exit from thread2

Thread 1:2

Thread 1:3

Thread 1:4

Exit from thread1

isAlive() Method

isAlive() method tests if the thread is alive or not and it return boolean value.

```
class MyThread extends Thread
{
public void run()
{
for(int i=0;i<5;i++)
{
System.out.println("status :"+isAlive());
}
}
}
```

```
class Demo
{
public static void main(String args[])
{
MyThread t1=new MyThread();
t1.start();
System.out.println("new status
:"+t1.isAlive());
}
}
```

output

```
new status :true
status :true
status :true
status :true
status :true
status :true
```

join() Method

- ❑ If a thread wants to wait until completing some other thread then we should go for join method.
- ❑ For example if a thread t1 wants to wait until completing t2 then “t1 has to call t2.join()”.
- ❑ If t1 executes t2.join() then immediately t1 will enter into waiting state until t2 completes.
- ❑ Once t2 completes then t1 will continue its execution.

EXAMPLE WITHOUT USING JOIN

```
class MyThread extends Thread
{
public void run()
{
for(int i=0;i<5;i++)
{
System.out.println("MyThread :"+i);
}
System.out.println("Exit from my thread");
}
}
```

```
class Demo
{
public static void main(String args[])
{
MyThread t1=new MyThread();
t1.start();
for(int j=0;j<5;j++)
{
System.out.println("Main Thread :"+j);
}
System.out.println("Exit from main thread");
}
}
```

Output

Main Thread :0

MyThread :0

Main Thread :1

Main Thread :2

MyThread :1

Main Thread :3

MyThread :2

Main Thread :4

MyThread :3

Exit from main thread

MyThread :4

Exit from my thread

EXAMPLE USING JOIN METHOD

```
class MyThread extends Thread
{
public void run()
{
for(int i=0;i<5;i++)
{
System.out.println("MyThread :"+i);
}
System.out.println("Exit from my thread");
}
}

class Demo
{
public static void main(String args[])
{
```

```
MyThread t1=new MyThread();
t1.start();
for(int j=0;j<5;j++)
{
try
{
t1.join();
}
catch(Exception e)
{
}
}

System.out.println("Exit from
main thread");
}
}
```

output

MyThread :0

MyThread :1

MyThread :2

MyThread :3

MyThread :4

Exit from my thread

Main Thread :0

Main Thread :1

Main Thread :2

Main Thread :3

Main Thread :4

Exit from main thread

Synchronization

- 🖱 **Synchronization in java is the capability of control the access of multiple threads to any shared resource.**
 - 🖱 **J a v a Synchronization is better option where we want to allow only one ~~had~~ to access the shared resource.**
 - 🖱 **W h y we use :**
 1. To prevent thread interference.
 2. To prevent consistency problem.
- Synchronized method is used to lock an object for any shared resource.**

Example without using Synchronized

```
class B extends Thread
{
    public void run()
    {
        for(int i=0;i<5;i++)
        {
            System.out.println(Thread.currentThread().getName()+(i));
        }
    }
}
```

```
class Demo
{
    public static void main(String args[])
    {
        B b=new B();
        Thread t1=new Thread(b);
        t1.setName("t1:");
        t1.start();
        Thread t2=new Thread(b);
        t2.setName("t2:");
        t2.start();
    }
}
```

output

t1:0

t1:1

t2:0

t1:2

t2:1

t1:3

t2:2

t1:4

t2:3

t2:4

Example using Synchronized

```
class B extends Thread
{
    public void run()
    {
        synchronized(this)
        {
            for(int i=0;i<5;i++)
            {
                System.out.println(Thread.currentThread().getName()+i);
            }
        }
    }
}
```

```
class Demo
{
    public static void main(String args[])
    {
        B b=new B();
        Thread t1=new Thread(b);
        t1.setName("t1:");
        t1.start();
        Thread t2=new Thread(b);
        t2.setName("t2:");
        t2.start();
    }
}
```

output

t1:0

t1:1

t1:2

t1:3

t1:4

t2:0

t2:1

t2:2

t2:3

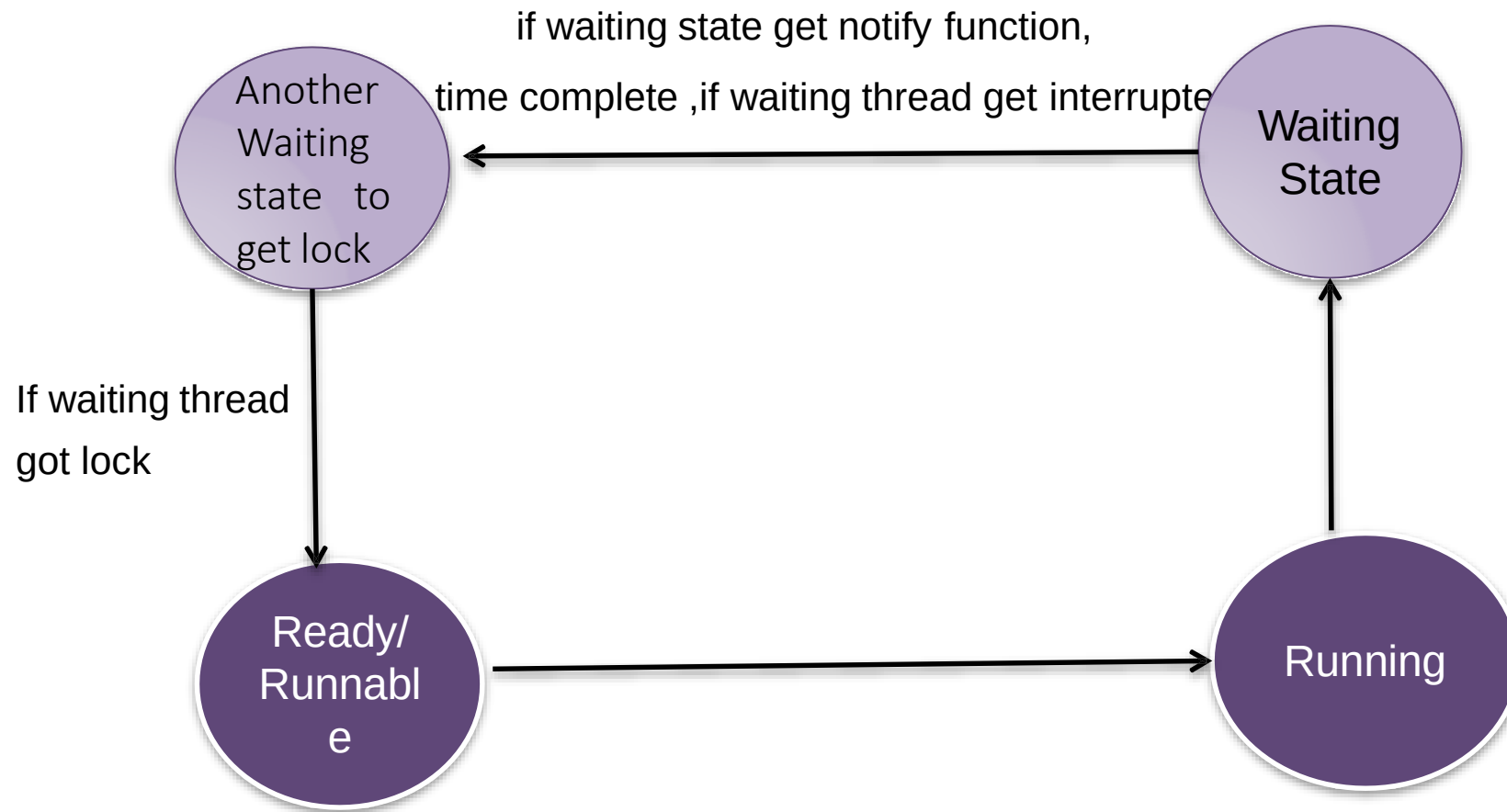
t2:4

wait() Method

- Two thread communicate each other using wait, notify and notifyAll methods.
- Wait, notify and notifyAll method are present in object class but not in thread class.
- Wait, notify and notifyAll method are called from synchronized area otherwise we get run time exception error saying “illegal monitor state exception”
- If a thread called wait method immediately enter in waiting state but in case of notify method it does not immediately enter in waiting state.
- Wait, notify and notifyAll methods are only method which release the lock.

Note:

Every wait method throws interrupted exception which is checked exception hence whenever we are using wait method compulsory we should handle these interrupted exception either by try, catch or by throws keyword otherwise we will get compile time error



EXAMPLE

```
class ThreadA
{
    public static void main(String args[]) throws Exception
    {
        ThreadB b=new ThreadB();
        b.start();
        synchronized(b)
        {
            System.out.println("main thread trying to call wait method"); //1//
            b.wait();
            System.out.println("main thread got notification"); //4//
            System.out.println(b.total); //5//
        }
    }
}
```

Thank You